

DAY 13

PANDAS FUNDAMENTALS

Detailed Beginner-to-Data-Science Study Guide

Muhammad Ahmad

60-Day Data Science Learning Journey

Goal	Move from NumPy arrays to real-world tabular data using Pandas.
Main structures	Series and DataFrame
Skills	Selection, filtering, sorting, inspection and basic missing-value handling
Project	Student DataFrame Analyzer

Learning path: Python → NumPy → Pandas → Data Cleaning → Data Visualization → Statistics → Machine Learning → Deep Learning

1. Day 13 Overview

Pandas is one of the most important Python libraries for Data Science. It helps you work with structured data such as CSV files, Excel files, databases and tables. Today we start from the basics and build toward a practical student-data analysis project.

By the end of Day 13:

- Explain what Pandas is and why it is used.
- Create and work with Series.
- Create DataFrames from dictionaries.
- Understand rows, columns, indexes, shape, size and data types.
- Select data with normal indexing, loc and iloc.
- Add, update and remove columns.
- Filter with one or multiple conditions.
- Sort and inspect datasets.
- Use info() and describe().
- Detect and handle basic missing values.
- Build a Student DataFrame Analyzer.

Prerequisite

You should know Python basics and the NumPy work from Days 11 and 12. Pandas uses many ideas you already learned, especially arrays, conditions and vectorized operations.

2. What Is Pandas?

Pandas is a Python library for data manipulation and analysis. It provides data structures that make tabular data easier to inspect, clean, transform and analyze.

Easy memory: NumPy is excellent for numerical arrays. Pandas is excellent for labeled tables.

Installation

```
pip install pandas
```

Importing Pandas

```
import pandas as pd

print(pd.__version__)
```

Line 1: imports Pandas and gives it the standard short name **pd**.

Line 3: displays the installed version.

3. Pandas Series

A Series is a one-dimensional labeled collection of values. Think of it as one column of a table with an index.

Create a Series

```
import pandas as pd

marks = pd.Series([80, 75, 90, 85])
print(marks)
```

0	80
1	75
2	90
3	85

```
dtype: int64
```

Pandas automatically creates indexes 0, 1, 2 and 3.

Custom Index

```
marks = pd.Series(  
    [80, 75, 90],  
    index=["Ali", "Usman", "Hamza"]  
)  
  
print(marks)  
  
Ali      80  
Usman    75  
Hamza    90  
dtype: int64
```

Access Values

```
print(marks["Ali"])  
print(marks.loc["Ali"])  
print(marks.iloc[0])  
  
80  
80  
80
```

loc selects by label. **iloc** selects by integer position.

Series Slicing

```
marks = pd.Series(  
    [80, 75, 90, 85],  
    index=["Ali", "Usman", "Hamza", "Ahmad"]  
)  
  
print(marks.iloc[0:2])  
print(marks.loc["Ali":"Hamza"])  
  
Ali      80  
Usman    75  
dtype: int64  
  
Ali      80  
Usman    75  
Hamza    90  
dtype: int64
```

iloc follows normal Python positional slicing. With **loc** and labels, the ending label can be included.

Series Calculations

```
marks = pd.Series([80, 75, 90, 85])  
  
print(marks.mean())  
print(marks.max())  
print(marks.min())  
print(marks.sum())  
  
82.5  
90  
75  
330
```

Series Filtering

```
marks = pd.Series([80, 75, 90, 85])  
print(marks[marks >= 80])  
  
0      80  
2      90  
3      85  
dtype: int64
```

The condition creates a Boolean mask. Pandas keeps only values where the mask is True.

4. Pandas DataFrame

A DataFrame is a two-dimensional labeled table. It is the main Pandas structure used for real-world datasets.

Create a DataFrame

```
import pandas as pd

data = {
    "name": ["Ali", "Usman", "Hamza", "Ahmad"],
    "age": [20, 21, 22, 20],
    "marks": [85, 72, 91, 78],
    "cgpa": [3.2, 3.0, 3.6, 3.1]
}

df = pd.DataFrame(data)
print(df)
```

	name	age	marks	cgpa
0	Ali	20	85	3.2
1	Usman	21	72	3.0
2	Hamza	22	91	3.6
3	Ahmad	20	78	3.1

Line-by-line: data creates a dictionary. Each key becomes a column. `pd.DataFrame(data)` converts it into a table. `print(df)` displays the result.

Shape

```
print(df.shape)

(4, 4)
```

The first number is rows and the second is columns.

Size

```
print(df.size)

16
```

$4 \times 4 = 16$ cells.

Columns, Index and Types

```
print(df.columns)
print(df.columns.tolist())
print(df.index)
print(df.dtypes)
```

`columns` gives column labels. `tolist()` converts them to a Python list. `index` describes row labels. `dtypes` tells you the type of each column.

5. Selecting Data

One Column

```
print(df["name"])
```

One column returns a Series.

Multiple Columns

```
print(df[["name", "marks"]])
```

	name	marks
0	Ali	85
1	Usman	72
2	Hamza	91
3	Ahmad	78

Double brackets are used because the inner part is a list of column names.

Rows With loc

```
print(df.loc[0])
print(df.loc[0:2, ["name", "marks"]])
```

The general form is **df.loc[rows, columns]**. loc is label-based.

Rows With iloc

```
print(df.iloc[0])
print(df.iloc[0:3, 0:3])
```

The general form is **df.iloc[row_positions, column_positions]**. iloc is position-based.

Tool	Uses	Example
loc	Labels	df.loc[0, "marks"]
iloc	Integer positions	df.iloc[0, 2]

6. Adding, Updating and Removing Data

Add a Column

```
df["status"] = "Pass"
```

Creates a new status column and puts Pass in every row.

Create Pass/Fail Automatically

```
import numpy as np

df["status"] = np.where(
    df["marks"] >= 50,
    "Pass",
    "Fail"
)
```

np.where(condition, true_value, false_value) creates a value based on a condition.

Update a Column

```
df["marks"] = df["marks"] + 5
```

Adds 5 to every mark.

Update One Student

```
df.loc[df["name"] == "Ali", "marks"] = 90
```

The condition finds Ali. The second label selects the marks column.

Remove a Column

```
df = df.drop(columns=["age"])
```

drop returns a modified DataFrame. Assigning it back to df stores the change in the current variable.

7. Filtering DataFrames

One Condition

```
high_marks = df[df["marks"] >= 80]
print(high_marks)
```

Keeps only rows where marks are at least 80.

Multiple Conditions: AND

```
result = df[
    (df["marks"] >= 70) &
    (df["cgpa"] >= 3.0)
]
print(result)
```

Use **&** for AND and put each condition inside parentheses.

Multiple Conditions: OR

```
result = df[
    (df["marks"] >= 90) |
    (df["cgpa"] >= 3.5)
]
```

Use **|** for OR. Do not use Python's and/or for element-wise Series conditions.

8. Sorting Data

```
print(df.sort_values("marks"))
print(df.sort_values("marks", ascending=False))
```

The first sorts lowest to highest. ascending=False sorts highest to lowest.

9. Inspecting a Dataset

head()

```
print(df.head())
print(df.head(2))
```

head() shows the first five rows by default. head(2) shows the first two.

tail()

```
print(df.tail())
print(df.tail(2))
```

tail() shows the last five rows by default. tail(2) shows the last two.

info()

```
df.info()
```

info() shows row count, column names, non-null counts, data types and memory usage. It is a useful first inspection command.

describe()

```
print(df.describe())
```

For numerical columns, describe() gives count, mean, standard deviation, minimum, quartiles and maximum.

10. Missing Values Basics

Real datasets often contain missing values. A missing value can occur because data was not supplied, not recorded, or was lost.

Create Missing Values

```
data = {
    "name": ["Ali", "Usman", "Hamza", "Ahmad"],
    "marks": [85, None, 91, 78],
    "cgpa": [3.2, 3.0, None, 3.1]
}

df = pd.DataFrame(data)
print(df)
```

None represents missing data here. Pandas often displays missing numerical values as NaN.

Detect Missing Values

```
print(df.isnull())
print(df.isnull().sum())
```

isnull() creates a True/False table. isnull().sum() counts missing values in each column.

Fill Missing Values

```
df["marks"] = df["marks"].fillna(  
    df["marks"].mean()  
)
```

The available marks are averaged, then the missing mark is replaced with that average.

Remove Rows With Missing Values

```
clean_df = df.dropna()
```

`dropna()` removes rows containing missing values. In real projects, choose the strategy based on the meaning of the data.

11. Practical Student Dataset

```
import pandas as pd  
import numpy as np  
  
data = {  
    "name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal", "Hassan"],  
    "age": [20, 21, 22, 20, 23, 21],  
    "department": ["CS", "CS", "SE", "CS", "SE", "CS"],  
    "marks": [85, 72, 91, 78, 65, 88],  
    "cgpa": [3.2, 3.0, 3.6, 3.1, 2.8, 3.4]  
}  
  
df = pd.DataFrame(data)  
print(df)
```

Average Marks

```
print(df["marks"].mean())  
  
79.83333333333333
```

Average CGPA

```
print(df["cgpa"].mean())  
  
3.1833333333333336
```

Highest Marks Student

```
top_student = df.loc[df["marks"].idxmax()]  
print(top_student)  
  
name           Hamza  
age            22  
department      SE  
marks          91  
cgpa           3.6  
Name: 2, dtype: object
```

`idxmax()` returns the index of the largest marks value. `loc` then retrieves that row.

Marks >= 80

```
print(df[df["marks"] >= 80])  
  
name  age  department  marks  cgpa  
0   Ali   20         CS     85   3.2  
2  Hamza   22         SE     91   3.6  
5  Hassan  21         CS     88   3.4
```

CS Students

```
print(df[df["department"] == "CS"])
```

Sort Highest First

```
print(df.sort_values("marks", ascending=False))
```

12. Complete Project: Student DataFrame Analyzer

Read the requirements before looking at the solution. Then type the solution yourself and change the values to test your understanding.

Requirements

- Create a student DataFrame.
- Display the complete dataset.
- Display first 3 students.
- Display last 2 students.
- Calculate average marks.
- Calculate average CGPA.
- Find the highest-marks student.
- Show students with marks ≥ 80 .
- Show only CS students.
- Sort by marks from highest to lowest.
- Add Pass/Fail status.
- Add grades using 90+ A+, 80–89 A, 70–79 B, 60–69 C, below 60 F.

Complete Code

```
import pandas as pd
import numpy as np

data = {
    "name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal", "Hassan"],
    "age": [20, 21, 22, 20, 23, 21],
    "department": ["CS", "CS", "SE", "CS", "SE", "CS"],
    "marks": [85, 72, 91, 78, 65, 88],
    "cgpa": [3.2, 3.0, 3.6, 3.1, 2.8, 3.4]
}

df = pd.DataFrame(data)

print("COMPLETE DATASET")
print(df)

print("\nFIRST THREE STUDENTS")
print(df.head(3))

print("\nLAST TWO STUDENTS")
print(df.tail(2))

average_marks = df["marks"].mean()
print("\nAVERAGE MARKS:", average_marks)

average_cgpa = df["cgpa"].mean()
print("AVERAGE CGPA:", average_cgpa)

top_student = df.loc[df["marks"].idxmax()]
print("\nHIGHEST MARKS STUDENT")
print(top_student)

print("\nSTUDENTS WITH MARKS  $\geq 80$ ")
print(df[df["marks"]  $\geq 80$ ])

print("\nCS STUDENTS")
print(df[df["department"] == "CS"])

print("\nSORTED BY MARKS")
print(df.sort_values("marks", ascending=False))

df["status"] = np.where(
```

```

    df["marks"] >= 50,
    "Pass",
    "Fail"
)

df["grade"] = np.where(
    df["marks"] >= 90, "A+",
    np.where(
        df["marks"] >= 80, "A",
        np.where(
            df["marks"] >= 70, "B",
            np.where(df["marks"] >= 60, "C", "F")
        )
    )
)

print("\nFINAL ANALYZED DATASET")
print(df)

```

Line-by-Line Project Explanation

1. import pandas as pd loads Pandas.
2. import numpy as np loads NumPy for np.where().
3. data stores the dataset in a dictionary.
4. pd.DataFrame(data) creates the table.
5. head(3) and tail(2) inspect selected rows.
6. mean() calculates averages.
7. idxmax() finds the highest marks row.
8. Boolean conditions filter records.
9. sort_values() orders the rows.
10. np.where() creates Pass/Fail.
11. Nested np.where() applies the grade rules from A+ down to F.

Expected Grade Result

Name	Marks	Status	Grade
Ali	85	Pass	A
Usman	72	Pass	B
Hamza	91	Pass	A+
Ahmad	78	Pass	B
Bilal	65	Pass	C
Hassan	88	Pass	A

13. Practice Tasks

Task 1

Create a Series with 80, 75, 90 and 95. Print it, mean, maximum and minimum.

Task 2

Create a custom-index Series for Ali, Usman, Hamza and Ahmad. Select Ali and Hamza by label and the first student with iloc.

Task 3

Create the four-column student DataFrame used in this lesson.

Task 4

Print shape, size, columns, index and dtypes.

Task 5

Print names, name plus marks, and the first two rows using loc and iloc.

Task 6

Find students with marks ≥ 80 .

Task 7

Find students with marks ≥ 70 AND CGPA ≥ 3.0 .

Task 8

Sort by marks ascending and descending.

Task 9

Create Pass/Fail using 50 as the passing mark.

Task 10

Build the complete Student DataFrame Analyzer.

14. Complete Solutions

Solution 1

```
import pandas as pd

marks = pd.Series([80, 75, 90, 95])

print(marks)
print("Mean:", marks.mean())
print("Maximum:", marks.max())
print("Minimum:", marks.min())

0    80
1    75
2    90
3    95
dtype: int64
Mean: 85.0
Maximum: 95
Minimum: 75
```

Solution 2

```
marks = pd.Series(
    [85, 72, 91, 78],
    index=["Ali", "Usman", "Hamza", "Ahmad"]
)

print(marks.loc["Ali"])
print(marks.loc["Hamza"])
print(marks.iloc[0])
```

Solution 3

```
data = {
    "name": ["Ali", "Usman", "Hamza", "Ahmad"],
    "age": [20, 21, 22, 20],
    "marks": [85, 72, 91, 78],
    "cgpa": [3.2, 3.0, 3.6, 3.1]
}

df = pd.DataFrame(data)
print(df)
```

Solution 4

```
print(df.shape)
print(df.size)
print(df.columns)
print(df.index)
print(df.dtypes)
```

Solution 5

```
print(df["name"])
print(df[["name", "marks"]])
print(df.loc[0:1])
print(df.iloc[0:2])
```

Solution 6

```
print(df[df["marks"] >= 80])
```

Solution 7

```
print(
    df[
        (df["marks"] >= 70) &
        (df["cgpa"] >= 3.0)
    ]
)
```

Solution 8

```
print(df.sort_values("marks"))
```

```
print(df.sort_values("marks", ascending=False))
```

Solution 9

```
import numpy as np

df["status"] = np.where(
    df["marks"] >= 50,
    "Pass",
    "Fail"
)

print(df)
```

Solution 10

Use the complete Student DataFrame Analyzer from Section 12. Re-type it rather than only copying it. Change student names, marks, departments and CGPAs to test whether your program still works.

15. Common Mistakes

Multiple-column selection

Wrong: `df["name", "marks"]`

Correct: `df[["name", "marks"]]`

Multiple conditions

Use parentheses around each condition: `(df["marks"] >= 70) & (df["cgpa"] >= 3.0)`

and/or confusion

Use `&` and `|` for element-wise Pandas conditions.

loc vs iloc

`loc` uses labels. `iloc` uses integer positions.

drop() misunderstanding

`drop` returns a new DataFrame unless you explicitly handle the operation differently.

Huge dataset printing

Use `head()`, `tail()`, `info()` and `describe()` instead of printing everything.

Missing values

Check `isnull().sum()` before doing analysis.

16. Quick Revision

Concept	Remember
Pandas	Library for data manipulation and analysis
Series	One-dimensional labeled data
DataFrame	Two-dimensional labeled table
shape	(rows, columns)
size	Total number of cells
columns	Column labels
index	Row labels
dtypes	Column data types
loc	Label-based selection
iloc	Integer-position selection
head()	First rows
tail()	Last rows
info()	Structure and non-null information
describe()	Numerical statistical summary
isnull()	Detect missing values
fillna()	Fill missing values
dropna()	Remove rows with missing values
sort_values()	Sort rows by a column

17. Final Checklist

- I can explain Pandas.
- I understand Series and DataFrames.
- I can create a DataFrame.
- I understand rows, columns and indexes.
- I can use shape, size, columns and dtypes.
- I can select columns and rows.
- I can use loc and iloc.
- I can add, update and remove columns.
- I can filter with one or multiple conditions.
- I can sort data.
- I can use head() and tail().
- I can use info() and describe().
- I can detect missing values.
- I understand basic fillna() and dropna().
- I can build the Student DataFrame Analyzer.

18. What You Learned Today

Day 13 is the transition from numerical array work to practical tabular data analysis. You learned how to represent data as Series and DataFrames, inspect its structure, select and filter records, modify columns, sort data, calculate statistics and handle basic missing values.

Next: keep practicing these operations until they feel natural. The next stages will build on this foundation with deeper data cleaning, grouping, aggregation, file workflows, visualization, statistics and Machine Learning.